МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «КИЇВСЬКИЙ АВІАЦІЙНИЙ ІНСТИТУТ»

Факультет комп'ютерних наук та технологій

Кафедра комп'ютеризованих систем управління

ЗВІТ

про виконання лабораторної роботи № 10

з дисципліни «Програмування на Python»

на тему: «Production Python: Docker, CI/CD та захист проєкту»

Варіант № 15: «Облік результатів наукових експериментів»

Виконав: студент Франков Д.

Перевірив: викладач

Київ — 2026

1. МЕТА РОБОТИ

Ознайомитися з передовими індустріальними підходами до підготовки, стандартизації та переведення Python-проєкту у повноцінний production-oriented стан; навчитися оформлювати проєкт як дистрибутивний інстальований пакет за стандартами PEP 517/PEP 621 (pyproject.toml); створювати дистрибутиви у форматах wheel та sdist; виносити параметри конфігурації у змінні середовища з використанням Pydantic Settings та запобігати збереженню секретів у репозиторії; проєктувати та оптимізувати multi-stage Dockerfile із захистом від root-доступу (non-root user appuser) та моніторингом через HEALTHCHECK; налаштовувати оркестрацію Docker Compose з persistent volume для надійного збереження даних; створювати наскрізний CI/CD pipeline на базі GitHub Actions для автоматизованого лінтингу (Ruff), статичної типізації (MyPy), тестування з фіксацією мінімального порогу покриття (pytest --cov-fail-under=85), збірки дистрибутивів та контейнерів; підготувати проєкт до фінального технічного захисту.

2. ПОСТАНОВКА ЗАДАЧІ ТА ОПИС ВАРІАНТА

Згідно з індивідуальним варіантом № 15 («Облік результатів наукових експериментів»), проєкт є фінальною консолідацією всіх рішень, напрацьованих протягом лабораторних робіт № 1–9. Предметна область охоплює облік наукових вимірювань (назва експерименту, дослідник, вимірюваний фізичний параметр, числовий результат і дата проведення).

Вимоги варіанта № 15 для лабораторної роботи № 10:

1. Створити production-орієнтований пакет з назвою experiment-registry (модуль experiment_registry);

2. Забезпечити збереження всієї предметної функціональності: реєстрація дослідників, створення експериментів, пошук, фільтрація за діапазоном, сортування за датою, розрахунок агрегованої статистики;

3. Реалізувати повнофункціональний REST API сервіс на базі FastAPI зі збереженням ендпоінтів /researchers, /experiments, /experiments/statistics та обов'язковим впровадженням ендпоінта моніторингу GET /health;

4. Використати SQLAlchemy 2.0 ORM та базу даних із конфігуруванням рядка підключення виключно через змінну середовища DATABASE_URL;

5. Створити клас Settings на базі pydantic-settings, шаблон .env.example та гарантувати ігнорування .env файлу через .gitignore;

6. Оформити метадані пакета у pyproject.toml, зібрати бінарний wheel (*.whl) та source distribution (*.tar.gz), перевірити встановлення та запуск CLI-команди experiment-registry у чистому віртуальному середовищі;

7. Створити Dockerfile (multi-stage build, non-root user, HEALTHCHECK) та docker-compose.yml з монтуванням persistent volume;

8. Налаштувати автоматизований workflow GitHub Actions (.github/workflows/ci.yml) із матрицею Python 3.11/3.12, перевірками ruff, mypy, pytest та build;

9. Скласти вичерпну документацію README.md, оформити виробничий чекліст та підготувати звіт з відповідями на всі контрольні запитання.

3. АРХІТЕКТУРА ТА СТРУКТУРА PRODUCTION-ПРОЄКТУ

Проєкт побудований за принципами Clean Architecture, слабкої зв'язності та високого зчеплення:

```
Lab_10/
├── .github/workflows/ci.yml    # Наскрізний CI/CD pipeline GitHub Actions
├── Dockerfile                  # Оптимізований multi-stage production Dockerfile
```

```
├── .dockerignore               # Виключення збіркових, тимчасових та секретних файлів
├── docker-compose.yml          # Оркестрація контейнера з persistent volume для БД
├── .env.example                # Публічний шаблон змінних оточення
├── .gitignore                  # Ігнорування .env, локальних БД, кешів, dist/
├── pyproject.toml              # Конфігурація пакета за стандартом PEP 517/621
├── README.md                   # Виробнича документація мікросервісу
├── LICENSE                     # Ліцензія MIT
├── alembic.ini                 # Конфігурація міграцій бази даних
├── generate_report.py          # Скрипт автоматичної генерації офіційного звіту
├── migrations/                 # Каталог версій міграцій схеми БД
├── src/
│   └── experiment_registry/    # Головний production-пакет
│       ├── __init__.py         # Версія пакета (1.0.0) та експорт фасадів
│       ├── config.py           # Pydantic Settings (DATABASE_URL, LOG_LEVEL тощо)
│       ├── logging_config.py   # Структуроване логування застосунку
│       ├── database.py         # Підключення до БД, сесії, engine, init_db
│       ├── models.py           # ORM моделі Researcher та Experiment
│       ├── schemas.py          # Pydantic v2 схеми валідації та серіалізації
│       ├── repositories.py     # Data Access Layer (CRUD, діапазони, агрегації)
│       ├── services.py         # Business Service Layer та аналітичні обчислення
│       ├── api.py              # FastAPI маршрутизатор та ендпоінт /health
│       └── main.py             # Точка входу CLI experiment-registry
└── tests/                      # Автоматизовані тести
    ├── conftest.py             # Фікстури in-memory SQLite та асинхронного клієнта
    ├── test_config.py          # Тестування налаштувань та перевантаження змінних
    ├── test_health.py          # Тести моніторингу здоров'я /health
    ├── test_api.py             # Інтеграційні тести REST API
    ├── test_repositories.py    # Тести репозиторіїв та моделей
    └── test_main.py            # Тестування параметрів CLI
```

4. ПАКЕТИЗАЦІЯ ТА ЗБІРКА (WHEEL ТА SDIST)

Пакетизація проєкту виконана згідно з сучасними специфікаціями PEP 517/621 у файлі pyproject.toml з використанням збіркового бекенду setuptools.build_meta. Визначено точку входу для консольної команди experiment-registry, що дозволяє запускати сервіс як стандартну утиліту операційної системи:

```
[build-system]
requires = ["setuptools>=70", "build"]
build-backend = "setuptools.build_meta"

[project]
name = "experiment-registry"
version = "1.0.0"
requires-python = ">=3.10"
```

```
dependencies = [
    "fastapi>=0.110.0", "uvicorn[standard]>=0.30.0",
    "sqlalchemy>=2.0.0", "pydantic>=2.7.0",
    "pydantic-settings>=2.0.0", "httpx>=0.27.0",
]
[project.scripts]
experiment-registry = "experiment_registry.main:main"
```

Збірка дистрибутивів виконана за допомогою утиліти build (python -m build). У каталозі dist/ згенеровано два дистрибутиви: бінарний wheel-пакет experiment_registry-1.0.0-py3-none-any.whl та вихідний архів experiment_registry-1.0.0.tar.gz. Працездатність wheel перевірено шляхом встановлення в ізольоване чисте віртуальне середовище: команда experiment-registry --version успішно повернула версію 1.0.0.

5. КОНФІГУРАЦІЯ ТА УПРАВЛІННЯ СЕКРЕТАМИ

Конфігурація застосунку реалізована за допомогою класу Settings, що успадковує BaseSettings із бібліотеки pydantic-settings. Параметри автоматично зчитуються зі змінних середовища операційної системи, а при локальній розробці — із файлу .env.

З метою захисту від витоку облікових даних (Secrets Management) реалізовано комплекс заходів:
1. Файл .env суворо внесено до .gitignore, що виключає його випадкову публікацію в системі контролю версій Git;
2. У репозиторії створено шаблонний файл .env.example, який містить лише опис змінних без реальних секретних значень;
3. Усі конфігураційні змінні мають безпечні дефолтні значення для локальної розробки (SQLite, debug logging);
4. Проведено аудит вихідного коду, Dockerfile та документації — захардкоджені паролі, токени та ключі доступу відсутні.

6. СПОСТЕРЕЖУВАНІСТЬ ТА МОНІТОРИНГ СТАНУ (/health)

Для забезпечення вимог моніторингу в хмарних середовищах та оркестраторах (Docker, Kubernetes) впроваджено ендпоінт GET /health, який виконує активну перевірку зв'язку з базою даних через тестовий запит SELECT 1 і повертає структурований JSON:

```
{
  "status": "ok",
  "app_name": "Scientific Experiments Registry",
  "environment": "production",
  "version": "1.0.0",
  "database": "connected"
}
```

Логування налаштовано через logging_config.py з динамічним вибором рівня (LOG_LEVEL) та структурованим форматуванням із зазначенням часової мітки, назви модуля та середовища запуску (env).

7. КОНТЕЙНЕРИЗАЦІЯ ЗА ДОПОМОГОЮ DOCKER

Для розгортання мікросервісу спроєктовано оптимізований multi-stage Dockerfile, що складається з двох етапів:
1. Builder stage (python:3.12-slim): копіювання вихідних файлів, збірка бінарного wheel за допомогою python -m build;
2. Runtime stage (python:3.12-slim): встановлення зібраного wheel із першого етапу, видалення тимчасових файлів, створення системного непривілейованого користувача appuser (UID 1000) та перемикання контексту USER appuser.

Застосування non-root користувача є критичною вимогою безпеки (Security Hardening), що запобігає можливості ескалації привілеїв на хост-системі у випадку компрометації застосунку. Також впроваджено інструкцію HEALTHCHECK для автоматичного опитування /health кожні 30 секунд. За

допомогою docker-compose.yml забезпечено моніторинг та збереження локальної бази даних у named volume experiment_data.

8. CI/CD PIPELINE ТА КОНТРОЛЬ ЯКОСТІ КОДУ

У каталозі .github/workflows/ci.yml реалізовано автоматизований пайплайн безперервної інтеграції (Continuous Integration), який автоматично запускається при кожному push або pull request до гілки main. Пайплайн містить такі обов'язкові етапи:

1. Lint check (ruff check Lab_10): перевірка дотримання правил стилю та виявлення потенційних дефектів;
2. Format check (ruff format --check Lab_10): перевірка відповідності форматування коду стандарту PEP 8;
3. Type check (mypy Lab_10/src): сувора статична типізація вихідного коду;
4. Automated testing & coverage (pytest -v --cov --cov-fail-under=85): запуск тестів із зупинкою при покритті нижче 85%;
5. Package build (python -m build Lab_10): верифікація збірки wheel та sdist із публікацією артефактів через upload-artifact;
6. Docker build: збірка виробничого Docker-образу з multi-stage Dockerfile.

9. РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА ПОКРИТТЯ

Усі 18 розроблених unit- та integration-тестів виконуються успішно за 0.38 секунди. Загальне покриття коду становить 92%, що суттєво перевищує встановлений норматив у 85%:

```
Name                                                    Stmts   Miss  Cover   Missing
-------------------------------------------------------------------------------------
Lab_10\src\experiment_registry\__init__.py                  8      0   100%
Lab_10\src\experiment_registry\api.py                      87      9    90%   33-38, 87-89
Lab_10\src\experiment_registry\config.py                   26      0   100%
Lab_10\src\experiment_registry\database.py                 26      6    77%   42-43, 48-52
Lab_10\src\experiment_registry\logging_config.py           13      0   100%
Lab_10\src\experiment_registry\main.py                     26     11    58%   47-58, 68
```

```
Lab_10\src\experiment_registry\models.py              33      0   100%
Lab_10\src\experiment_registry\repositories.py        86      2    98%   113, 149
Lab_10\src\experiment_registry\schemas.py             76      6    92%   102-107
Lab_10\src\experiment_registry\services.py            57      2    96%   86, 93
-------------------------------------------------------------------------------
TOTAL                                                438     36    92%

============================== 18 passed in 0.38s ==============================
```

10. ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке Python package?

Python package — це структурований спосіб організації простору імен модулів за допомогою директорій, що містять файл __init__.py (або згідно з PEP 420 namespace packages), що дозволяє імпортувати функції та класи через крапкову нотацію.

2. Що таке distribution package?

Distribution package — це зібраний і запакований архів файлів (наприклад, wheel або tar.gz sdist) із версіонованими метаданими, призначений для публікації в індексах пакетів (PyPI) та встановлення менеджерами pip або poetry.

3. Що таке wheel?

Wheel (PEP 427) — це стандартизований готовий до встановлення бінарний формат дистрибутиву Python (ZIP-архів), що містить попередньо скомпільований код та метадані, не вимагаючи повторної збірки при інсталяції.

4. Яке розширення має wheel?

Файли формату wheel мають стандартне розширення .whl.

5. Що таке sdist?

Source Distribution (sdist) — це архів вихідного коду проєкту (зазвичай у форматі .tar.gz), що містить вихідні тексти, файли збірки, ліцензію та документацію, з якого менеджер pip може виконати збірку wheel безпосередньо на цільовій машині.

6. Для чого використовується pyproject.toml?

Файл pyproject.toml (PEP 517/518/621) є єдиним декларативним стандартом для опису збіркових вимог проєкту, інструментів збірки (build-backend), метаданих застосунку (версія, автори, залежності, точки входу) та конфігурацій лінтерів і тестерів.

7. Що містить [build-system]?

Секція [build-system] визначає перелік пакетів, необхідних для збірки проєкту (наприклад, requires = ['setuptools>=70', 'build']), та назву програмного модуля збіркового бекенду (build-backend = 'setuptools.build_meta').

8. Що містить [project]?

Секція [project] містить стандартизовані метадані дистрибутиву: назву пакета (name), версію (version), опис, вимоги до версії інтерпретатора (requires-python), основні залежності (dependencies), опціональні залежності (optional-dependencies) та консольні команди (scripts).

9. Як створити wheel?

За допомогою стандартної утиліти build, виконавши у терміналі команду: python -m build --wheel.

10. Як створити sdist?

Виконавши команду збірки вихідного дистрибутиву: python -m build --sdist.

11. Для чого використовується python -m build?

Модуль build запускає ізольований процес збірки дистрибутивних пакетів (wheel та sdist) за стандартом PEP 517, автоматично встановлюючи необхідний build-backend у тимчасове віртуальне середовище.

12. Що таке semantic versioning?

Семантичне версіонування (SemVer, semver.org) — це система формалізації номерів релізів у вигляді MAJOR.MINOR.PATCH, що однозначно інформує користувачів про рівень сумісності змін.

13. Що означає MAJOR?

Номер MAJOR збільшується при внесенні суттєвих змін, які ламають зворотну сумісність публічного API (breaking changes).

14. Що означає MINOR?

Номер MINOR збільшується при додаванні нової функціональності зі збереженням повної зворотної сумісності.

15. Що означає PATCH?

Номер PATCH збільшується при виправленні помилок і дефектів (bug fixes) без додавання нової функціональності та зі збереженням зворотної сумісності.

16. Що таке environment variable?

Змінна середовища (Environment Variable) — це іменоване динамічне значення операційної системи, доступне процесам для налаштування їхньої поведінки без зміни вихідного коду.

17. Навіщо configuration виносити з коду?

Винесення конфігурації відповідає III фактору методології The Twelve-Factor App, дозволяючи запускати однаковий скомпільований код у різних оточеннях (development, testing, staging, production) та захищає чутливі дані від потрапляння у відкритий код.

18. Що таке secret?

Secret (секрет) — це конфіденційні цифрові дані авторизації та аутентифікації, які надають доступ до критичних ресурсів системи (паролі БД, API-ключі, приватні сертифікати, токени доступу).

19. Наведіть приклади secrets.

DATABASE_PASSWORD, SECRET_KEY для підпису JWT-токенів, AWS_SECRET_ACCESS_KEY, приватні SSH-ключі, Stripe API Token.

20. Чому secrets не можна commit у Git?

Тому що система контролю версій Git зберігає повну історію змін, доступну всім учасникам проєкту або публічно на GitHub, що призводить до компрометації інфраструктури, несанкціонованого доступу та витоку персональних даних.

21. Для чого використовується .env?

Файл .env використовується для локального збереження змінних середовища у форматі KEY=VALUE під час розробки, які автоматично підвантажуються бібліотеками python-dotenv або pydantic-settings.

22. Чому .env має бути у .gitignore?

Оскільки файл .env містить реальні локальні облікові записи, паролі та секрети, його обов'язково вносять до .gitignore, щоб запобігти випадковій передачі у віддалений Git-репозиторій.

23. Для чого потрібний .env.example?

Файл .env.example слугує публічною документацією конфігурації проєкту: він містить перелік усіх обов'язкових ключів із фіктивними значеннями або порожніми рядками без жодних реальних секретів.

24. Що таке Docker?

Docker — це відкрита платформа контейнеризації на базі можливостей ядра Linux (cgroups, namespaces), яка дозволяє автоматизувати пакування, ізоляцію, доставку та запуск програмного забезпечення у стандартизованих контейнерах.

25. Що таке Dockerfile?

Dockerfile — це текстовий скрипт з інструкціями (FROM, COPY, RUN, CMD тощо), які послідовно виконуються Docker-демоном для автоматичної збірки незмінного образу (Docker image).

26. Що таке Docker image?

Docker image — це незмінний (read-only) шаблон з кількох нашарованих файлових систем, що містить операційну систему, середовище виконання, бібліотеки та код застосунку, необхідні для запуску.

27. Що таке Docker container?

Docker container — це запущений ізольований екземпляр Docker image, що має власний тонкий шар для запису (read-write layer), власні мережеві інтерфейси та виділені системні ресурси.

28. Чим image відрізняється від container?

Image — це статичний образ (пасивний кресленик або шаблон програми), а Container — це активний процес виконання цього образу в ізольованому середовищі ОС.

29. Для чого використовується docker build?

Команда docker build зчитує інструкції Dockerfile та контекст збірки для створення нового Docker image на локальній машині.

30. Для чого використовується docker run?

Команда docker run створює новий контейнер на основі вказаного образу та запускає його на виконання.

31. Що робить EXPOSE?

Інструкція EXPOSE документує мережевий порт, який слухає контейнер під час роботи (слугує метаданими для адміністраторів і docker run -P).

32. Що робить CMD?

Інструкція CMD задає команду за замовчуванням та її аргументи, які виконуються при старті контейнера.

33. Для чого потрібний WORKDIR?

Інструкція WORKDIR встановлює поточну робочу директорію для всіх наступних інструкцій RUN, CMD, ENTRYPOINT, COPY та ADD у Dockerfile.

34. Що робить COPY?

Інструкція COPY копіює локальні файли чи каталоги з контексту збірки хост-системи у файлову систему контейнера за вказаним шляхом.

35. Для чого використовується .dockerignore?

Файл .dockerignore запобігає копіюванню непотрібних, великих або секретних файлів (.git, .env, __pycache__, tests, venv) у контекст збірки Docker, що значно прискорює збірку та зменшує розмір фінального образу.

36. Що таке Docker volume?

Docker volume — це механізм постійного збереження даних (persistent storage) за межами файлової системи контейнера, що дозволяє зберігати бази даних та файли навіть після видалення чи перезапуску контейнера.

37. Для чого потрібний health endpoint?

Health endpoint (GET /health) дозволяє системам балансування навантаження, оркестраторам (Kubernetes, Docker Swarm) та інструментам моніторингу автоматично перевіряти працездатність застосунку та доступність залежностей (БД, кеш).

38. Що таке CI?

Continuous Integration (CI, неперервна інтеграція) — це практика автоматичного регулярного злиття змін розробників у спільну гілку репозиторію з обов'язковим автоматичним запуском лінтерів, типізаторів, тестів та збірок для раннього виявлення дефектів.

39. Що таке CD?

Continuous Delivery / Deployment (CD, неперервна доставка/розгортання) — це автоматизований процес доставки перевіреного коду в тестове (staging) або бойове (production) середовище без участі людини або за одним кліком.

40. Що таке GitHub Actions?

GitHub Actions — це інтегрована в GitHub платформа автоматизації робочих процесів (CI/CD), що дозволяє виконувати скрипти та завдання при виникненні подій у репозиторії (push, pull request, release).

41. Що таке workflow?

Workflow — це автоматизована процедура, що описується у YAML-файлі в каталозі .github/workflows/ та складається з одного чи декількох завдань (jobs).

42. Що таке job?

Job — це сукупність послідовних кроків (steps), які виконуються на одному спільному віртуальному сервері або контейнері (runner).

43. Що таке step?

Step — це окрема інструкція або дія (запуск shell-команди або готовий модуль action), що виконується в межах завдання job.

44. Що таке runner?

Runner — це фізичний або хмарний сервер (наприклад, віртуальна машина ubuntu-latest на серверах GitHub), на якому запускається та виконується завдання job.

45. Коли повинен запускатися CI?

CI повинен запускатися автоматично при кожному push у головні гілки, при створенні та кожному оновленні Pull Request, а також періодично за розкладом (nightly builds) для виявлення застарілих залежностей.

46. Для чого потрібний lint?

Лінтер (Linter) виконує статичний аналіз вихідного коду на відповідність стандартам оформлення (PEP 8), виявляє синтаксичні огріхи, невикористані імпорти, потенційні помилки логіки та вразливості безпеки.

47. Для чого використовується Ruff?

Ruff — це ультрашвидкий сучасний лінтер і форматер для Python, написаний на Rust, який замінює одночасно Flake8, Black, isort, pyupgrade та pydocstyle, виконуючи перевірки у 10–100 разів швидше за традиційні інструменти.

48. Для чого потрібний mypy?

MyPy — це статичний аналізатор типів для Python, який перевіряє правильність використання анотацій типів (Type Hints, PEP 484) до запуску програми, запобігаючи помилкам TypeError та помилкам атрибутів.

49. Що таке type checking?

Type checking (перевірка типів) — це процес верифікації відповідності типів аргументів функцій, повернених значень та змінних визначеним декларативним типам під час компіляції або статичного аналізу.

50. Навіщо tests запускати в CI?

Щоб гарантувати, що новий код не ламає існуючу функціональність (регресія), не вносить багів та задовольняє всім бізнес-вимогам перед потраплянням у спільну кодову базу.

51. Навіщо package build виконувати в CI?

Щоб гарантувати, що конфігурація pyproject.toml коректна, всі файли правильно включені в дистрибутив і проєкт гарантовано збирається у стандартний wheel без локальних артефактів розробника.

52. Навіщо Docker build виконувати у CI?

Щоб переконатися, що Dockerfile синтаксично правильний, усі інструкції успішно виконуються на чистій базовій ОС, залежності завантажуються, а фінальний образ готовий до розгортання.

53. Що таке GitHub Secret?

GitHub Secret — це зашифрована змінна оточення, збережена в налаштуваннях безпеки репозиторіїв GitHub, яка безпечно передається у CI/CD раннери без відображення у відкритих логах (маскується як ***).

54. Як використовувати secret у workflow?

За допомогою синтаксису виразів: ${{ secrets.MY_SECRET_NAME }}, передаючи значення в секцію env кроку workflow.

55. Що таке documentation?

Документація — це текстовий та графічний опис архітектури, вимог, процедур встановлення, налаштування та використання програмного забезпечення для користувачів та інших розробників.

56. Що повинен містити README?

README повинен містити назву проєкту, опис функціоналу, системні вимоги, інструкцію зі швидкого старту (встановлення, запуск у venv та Docker), опис змінних середовища, специфікацію API, команди тестування та ліцензію.

57. Для чого потрібні docstrings?

Docstrings (рядки документації за стандартом PEP 257) вбудовуються у модулі, класи та функції для пояснення їхнього призначення, параметрів і результатів, доступні через функцію help() та використовуються генераторами документації.

58. Що таке OpenAPI?

OpenAPI (раніше Swagger) — це загальноприйнятий машиночитаний стандарт опису інтерфейсів REST API у форматі JSON або YAML.

59. Що таке /docs у FastAPI?

Маршрут /docs у FastAPI надає автоматично згенерований інтерактивний веб-інтерфейс Swagger UI для тестування ендпоінтів безпосередньо з браузера.

60. Що таке /openapi.json?

Ендпоінт /openapi.json повертає сиру специфікацію API у форматі схеми OpenAPI JSON, що використовується для генерації клієнтських бібліотек та автодокументації.

61. Що таке logging?

Логування — це процес реєстрації системних подій, повідомлень та помилок програми у структурованому текстовому форматі із зазначенням часу, рівня важливості та джерела події.

62. Чому logging краще за print?

Тому що логування дозволяє гнучко фільтрувати події за рівнями (DEBUG, INFO, ERROR), спрямовувати вивід у різні канали (консоль, файл, syslog, ELK), містить контекстні метадані (час, потік, модуль) та не сповільнює production-код.

63. Що таке production configuration?

Це набір налаштувань застосунку, оптимізований для максимальної надійності, швидкодії та безпеки у робочому середовищі: відключено debug-режим, увімкнено рівень логів INFO/WARN, використовуються стійкі СУБД та суворі CORS-правила.

64. Чим development відрізняється від production?

Development орієнтований на зручність налагодження розробника (auto-reload, детальний DEBUG вивід, in-memory чи локальні БД), а Production орієнтований на стабільність, стійкість до відмов, безпеку секретів та високу продуктивність.

65. Що таке Docker healthcheck?

Це вбудований механізм Docker daemon (інструкція HEALTHCHECK), який періодично запускає вказану команду всередині контейнера для перевірки його працездатності та присвоює контейнеру статус healthy або unhealthy.

66. Для чого потрібний non-root user?

Запуск процесів всередині контейнера від імені non-root користувача (appuser) зменшує поверхню атаки: у разі зламу застосунку зловмисник не отримає прав суперкористувача root на хостовій операційній системі.

67. Що таке multi-stage build?

Multi-stage build — це техніка створення Dockerfile з кількома інструкціями FROM, де в перших етапах компілюються та збираються артефакти з важкими компіляторами, а у фінальний мінімальний образ копіюються лише готові бінарні файли без важкого інструментарію збірки.

68. Що таке coverage?

Code Coverage (покриття коду) — це метрика, яка виражає у відсотках частку рядків і розгалужень вихідного коду програми, які були виконані під час прогону автоматизованих тестів.

69. Навіщо встановлювати coverage threshold?

Встановлення порогового значення (--cov-fail-under=85) унеможливлює злиття нового коду без відповідних тестів, автоматично завершуючи пайплайн помилкою у разі падіння якості.

70. Що означає green CI pipeline?

Green CI pipeline означає, що всі перевірки якості коду (лінтер, форматер, типізатор), усі тести, збірка дистрибутиву та збірка Docker-образу завершилися зі статусом успіху (exit code 0).

11. ВИСНОВОК

У ході виконання лабораторної роботи № 10 було повністю завершено розробку наскрізного індивідуального проєкту системи обліку результатів наукових експериментів (Варіант № 15) та переведено його у повноцінний production-oriented стан.

У результаті роботи:

1. Оформлено дистрибутивний пакет experiment-registry за стандартами PEP 517/621 у файлі pyproject.toml, зібрано wheel та sdist дистрибутиви, верифіковано встановлення у чистому оточенні;

2. Реалізовано централізовану систему конфігурації за допомогою Pydantic Settings з підтримкою змінних середовища та захистом від витоку секретів;

3. Створено ендпоінт моніторингу GET /health та налаштовано структуроване логування;

4. Розроблено безпечний multi-stage Dockerfile із запуском від non-root користувача appuser, вбудованим HEALTHCHECK та оркестрацією docker-compose з persistent volumes;

5. Налаштовано надійний CI/CD pipeline на базі GitHub Actions для автоматичної перевірки Ruff, MyPy, Pytest (покриття 92%) та збірки образів;

6. Сформовано повний виробничий чекліст та вичерпні відповіді на всі 70 контрольних запитань.